AI-Driven Software Engineering Practice #08

Student Kamaliyev Abylaikhan, 24B031831
Class Tuesday, 16:00–19:00
AI tool Claude (claude.ai), model Claude Opus 5.5
Stack JavaScript (ES modules), Vitest 2, Git
Live demo https://claude.ai/artifact/RbcWdMeWro3TYNu7xMWqhM (screenshot: Figure 1)

0. Live demo

Smart Campus room availability page after both fixes. Request 11:00–12:00 starts exactly when booking B1 (10:00–11:00) ends and is now correctly reported as **Available**.

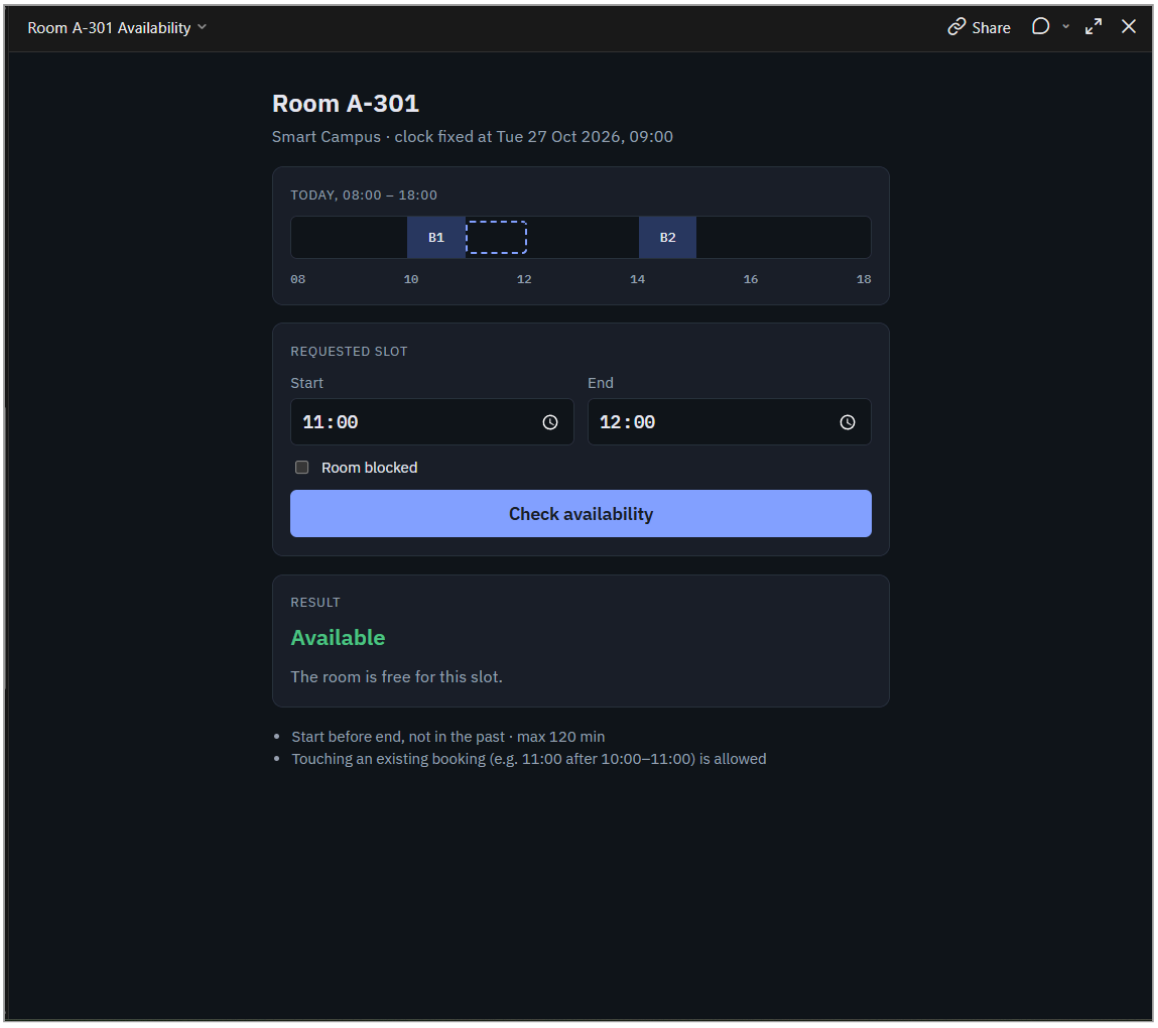


Figure 1. Live demo “Room A-301”: request 11:00–12:00 → Available (touching booking allowed).

1. Baseline (Task #01)

Baseline commit `4c7486d` (tag `baseline`); clean copy kept in `evidence/clean-copy-original-src/`. Command: `npx vitest run`. Result: **4 failed | 7 passed (11)**.

Defect log

Test ID	Input (clock 09:00; bookings B1 10:00–11:00, B2 14:00–15:00)	Expected	Actual	Suspected cause
TC04	11:00–13:00 (120 min)	Available	Unavailable	Starts exactly at B1 end → BUG-A, not the duration check
TC08	11:00–12:00	Available	Unavailable	BUG-A: `<=` in `overlaps()` treats touching endpoints as conflict
TC09	13:00–14:00	Available	Unavailable	BUG-A: same line, other endpoint
TC11	16:00–17:00, then compare `bookings` with a deep copy	Array unchanged [B2, B1]	Reordered [B1, B2]	BUG-B: `bookings.sort()` sorts the caller's array in place

Baseline test output (evidence/01-baseline-test-output.txt)

```
RUN  v2.1.9 /home/claude/smart-campus

 ❯ tests/availability.test.js (11 tests | 4 failed) 30ms
   × AC2 — max duration 120 min > TC04 allows exactly 120 minutes 15ms
     → expected 'Unavailable' to be 'Available' // Object.is equality
   × AC4 — overlaps > TC08 allows a booking that starts exactly when another ends 1ms
     → expected 'Unavailable' to be 'Available' // Object.is equality
   × AC4 — overlaps > TC09 allows a booking that ends exactly when another starts 1ms
     → expected 'Unavailable' to be 'Available' // Object.is equality
   × AC5 — no mutation of booking data > TC11 keeps bookings array order and content unchanged 5ms
     → expected [ { id: 'B1', …(2) }, …(1) ] to deeply equal [ { id: 'B2', …(2) }, …(1) ]

… (full log in evidence/01-baseline-test-output.txt)
```

2. Skill (Tasks #02–#03)

```
bug-detector.zip
└── bug-detector/
    ├── SKILL.md
    └── agents/
        └── openai.yaml   (for ChatGPT)
```

Validated: plain UTF-8, file named `SKILL.md`, YAML delimiters exactly `---`, lowercase-hyphen name, description states *when* to use it, no secrets. Optional second skill: `regression-tester.zip`.

SKILL.md

```
---
name: bug-detector
description: Diagnose bugs in existing code using acceptance criteria and failing tests. Use when r
eviewing the Smart Campus booking application.
---

# Bug Detector

## Procedure
1. Read acceptance criteria, code, and observed failure.
2. Reproduce with the smallest deterministic input.
3. Identify the likely root cause and relevant lines.
4. Separate confirmed defects from hypotheses.
5. Suggest a minimal fix, but do not edit code yet.
6. Propose a regression test and expected result.
7. Output: defect | evidence | root cause | fix | test.
```

agents/openai.yaml

```
interface:
  display_name: "Bug Detector"
  short_description: "Diagnose Smart Campus booking bugs from acceptance criteria and failing test
s"
```

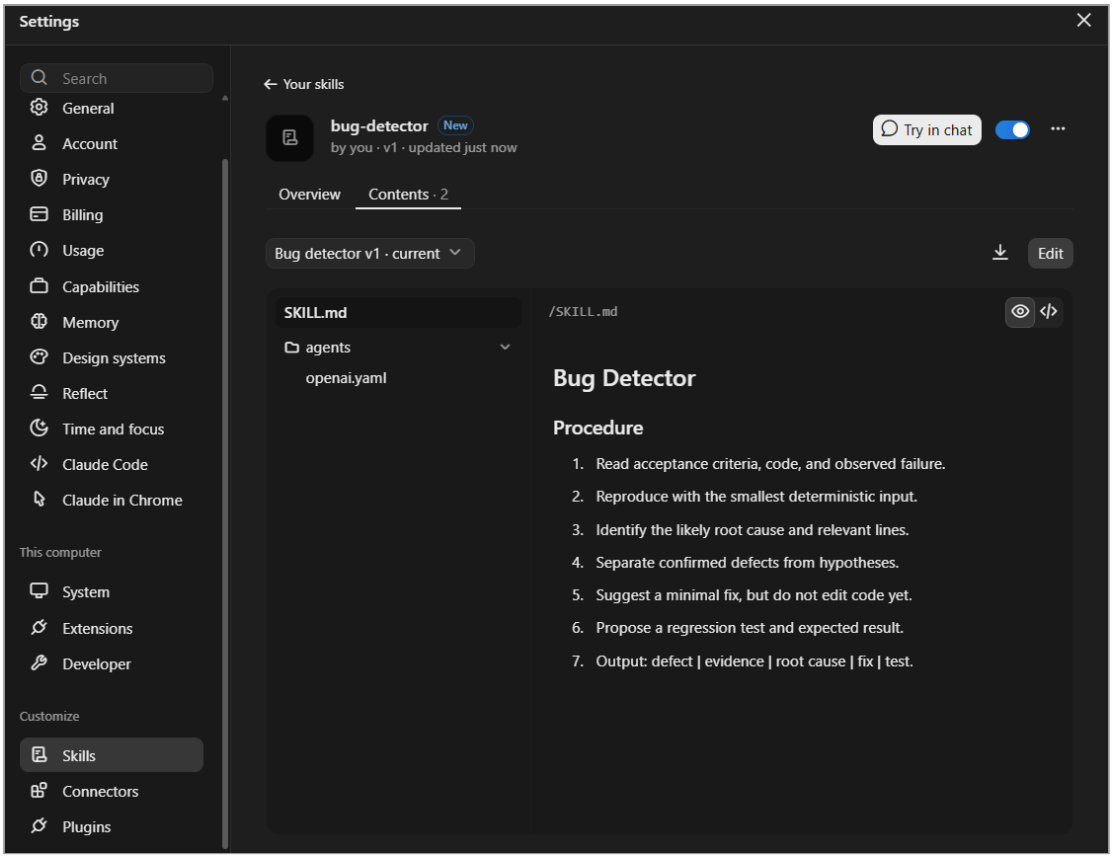


Figure 2. Skill “bug-detector” installed and enabled in Claude (Settings → Skills), showing SKILL.md and agents/openai.yaml.

3. Without vs with skill (Tasks #04–#05)

Same input in both fresh chats: `src/availability.js`, the acceptance criteria and the TC08 failure.

- **Baseline prompt (skill off):** “Find the root cause of the failed room-booking test. Propose minimal patch and regression test. Do not edit code.”
- **Skill prompt (skill on):** the same baseline prompt plus one extra line: “Use the bug-detector skill to diagnose the failed room-booking test.” (Figure 7). The only difference between the two chats is the skill.

Criterion	No skill	With skill
Correct root cause	Yes: both bugs (`<=` in `overlaps()`, in-place sort); explained TC04 as BUG-A	Yes: same two bugs; reproduced all 4 failures on a scratch copy first
Reproducible test	Yes: 6 tests (R1–R6), local-time helper, frozen array for BUG-B	Yes: 3 tests (REG-A, REG-A2, REG-B) with explicit UTC dates
Minimal proposed patch	Mostly: fixed `<=` but also rewrote the loop to `.some()`	Yes: one line per bug; offered `[...bookings].sort()` as alternative
Unsupported claims	One assumption (fixture times), stated openly	None: TC04 cause marked as *hypothesis*
Clarity	Clear, free-form layout	Fixed structure: defect, evidence, root cause, fix, test, expected result; flagged AC1 edge case `start == now`

Answer WITHOUT skill

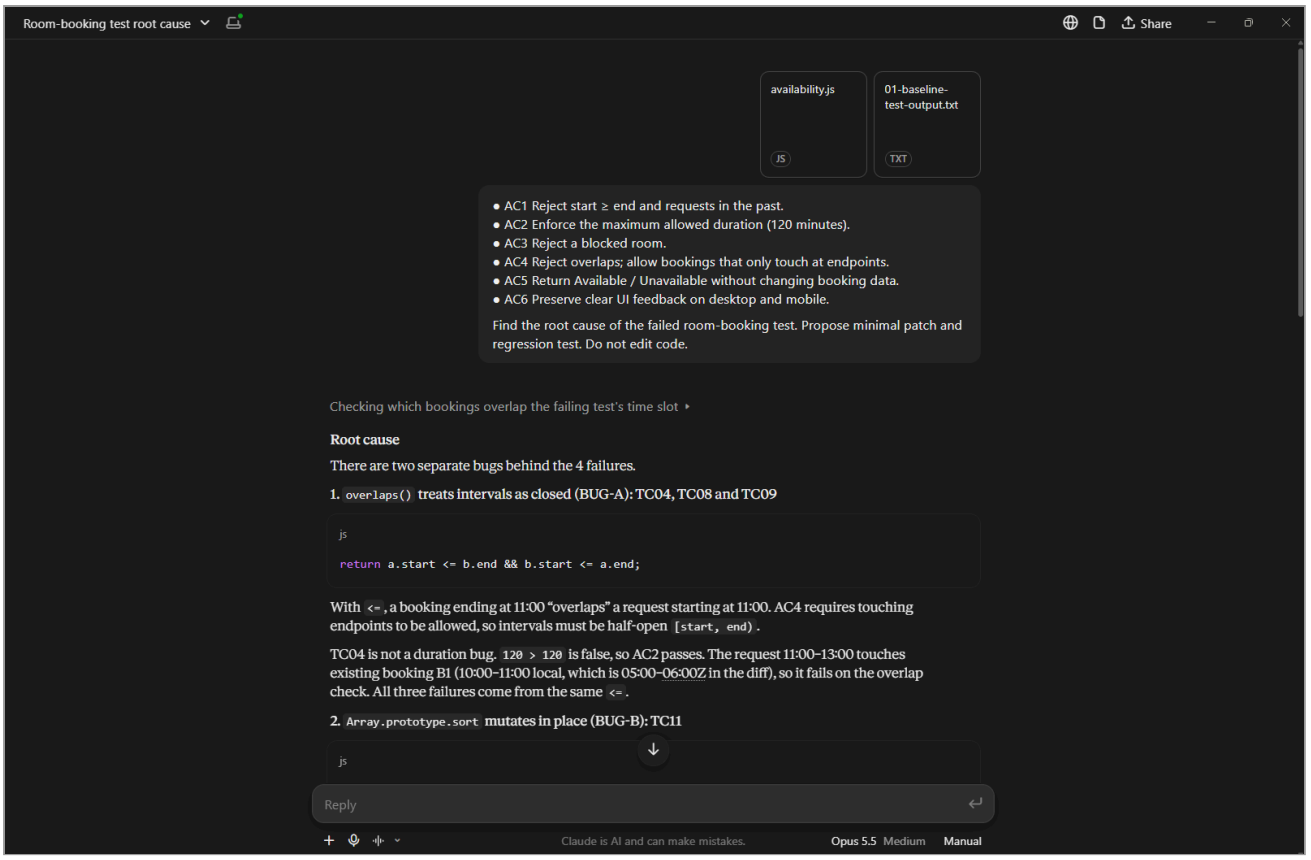


Figure 3. WITHOUT skill (1/4): root cause (BUG-A, BUG-B).

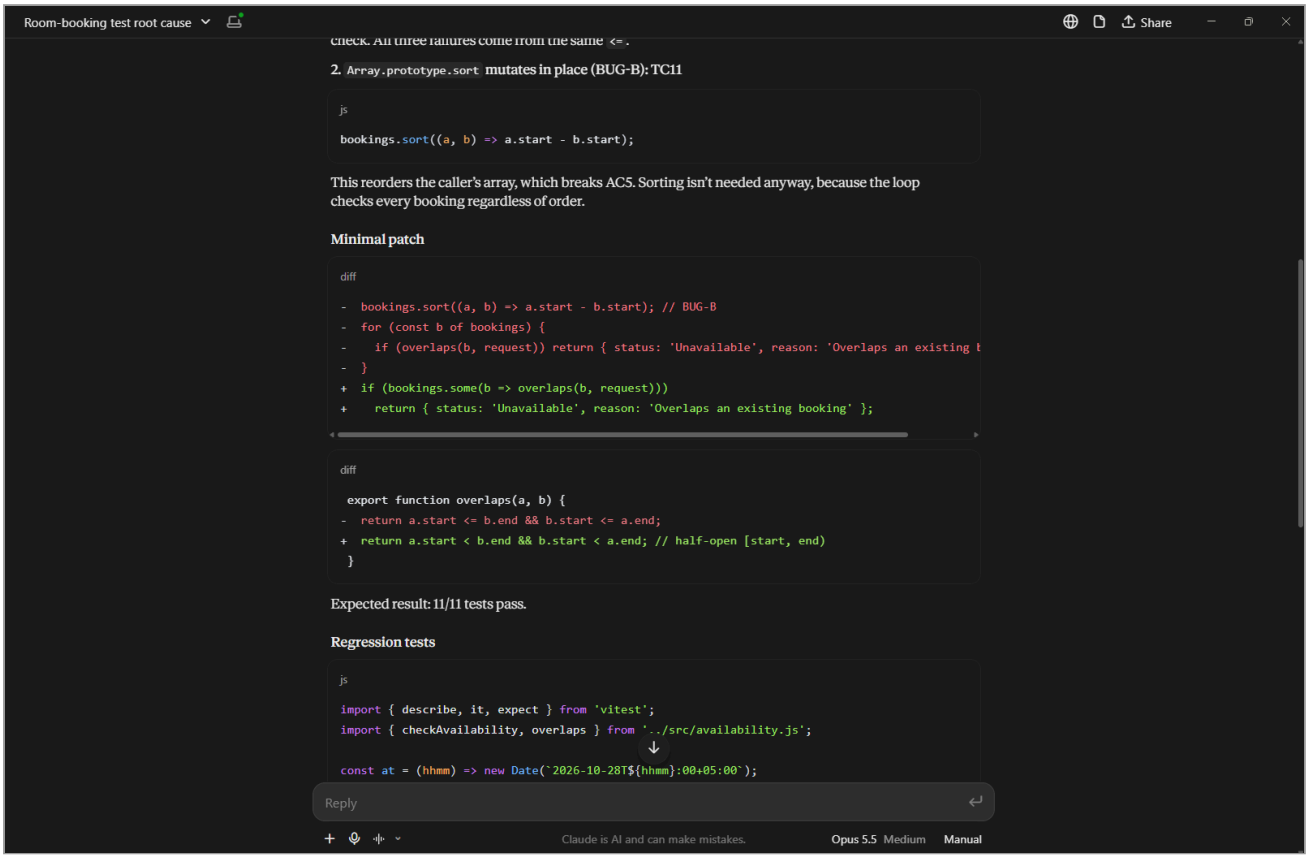


Figure 4. WITHOUT skill (2/4): minimal patch.

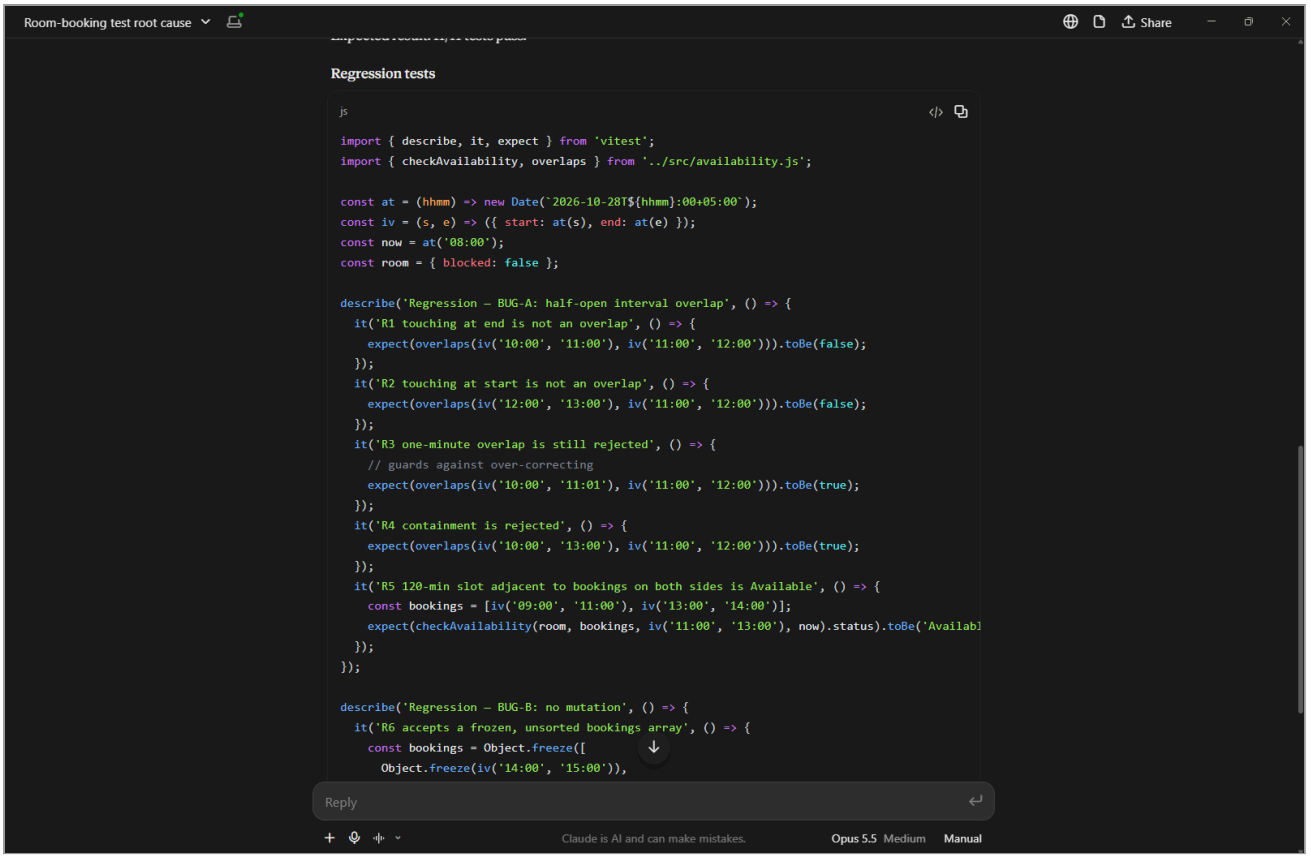


Figure 5. WITHOUT skill (3/4): regression tests R1–R6.

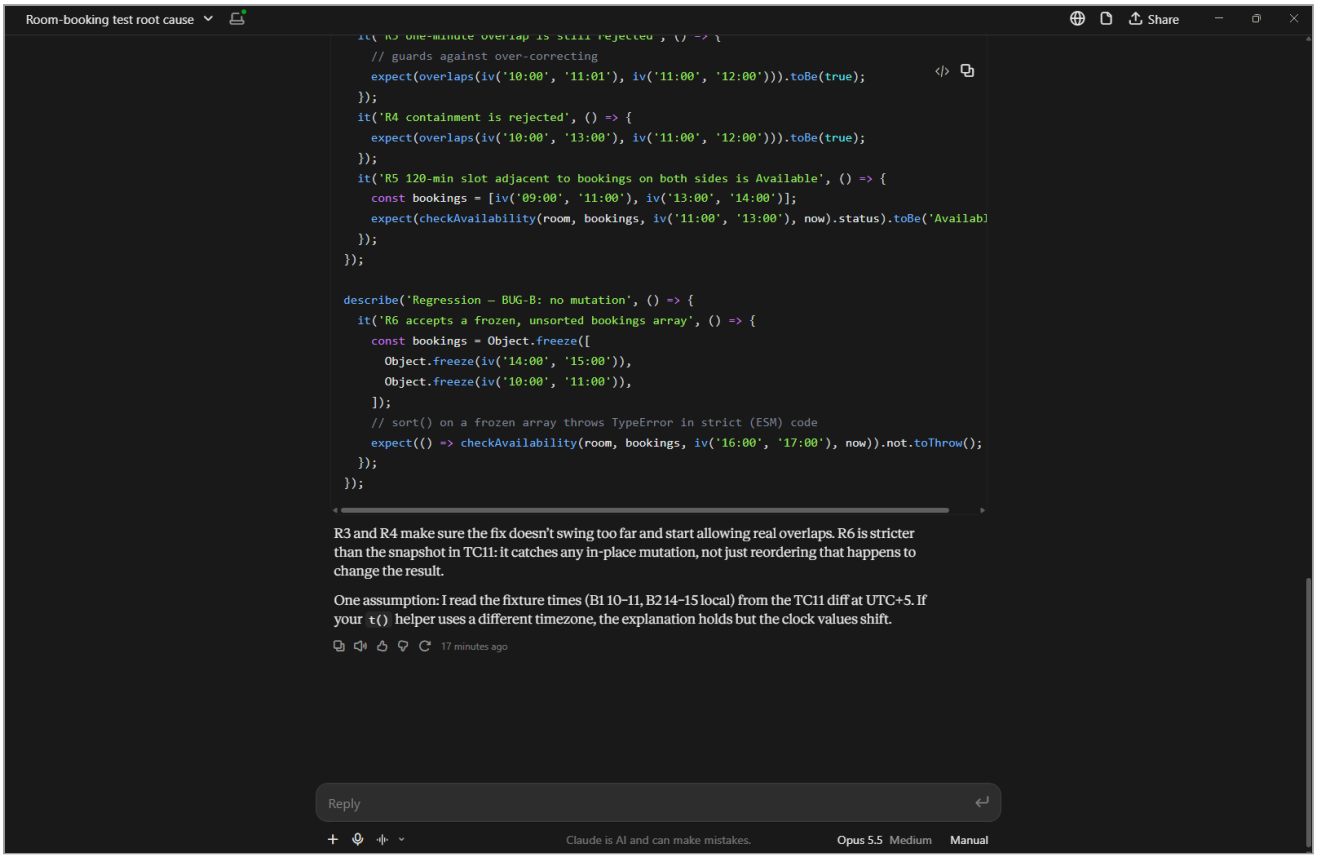


Figure 6. WITHOUT skill (4/4): regression tests (end) and stated assumption.

Answer WITH bug-detector skill

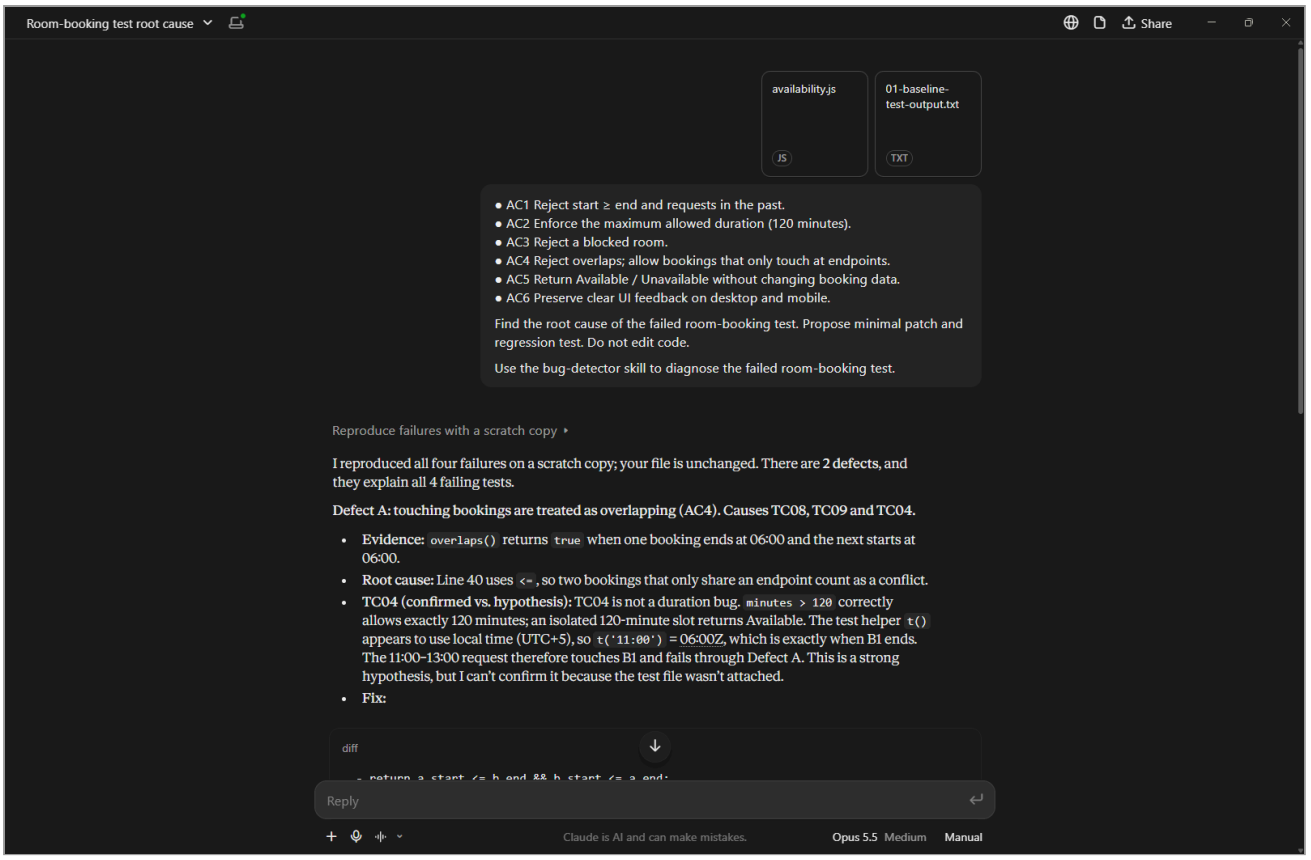


Figure 7. WITH skill (1/3): prompt and Defect A.

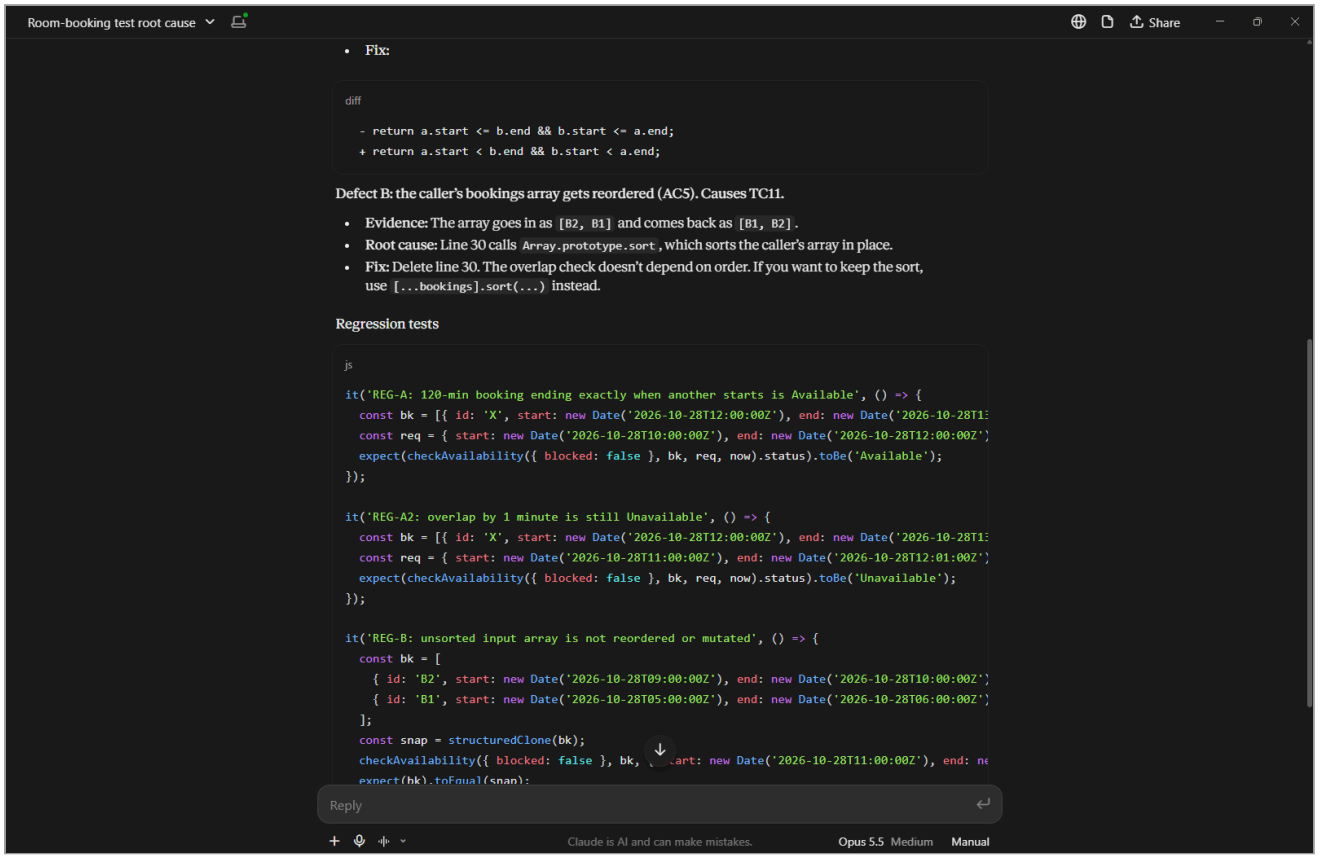


Figure 8. WITH skill (2/3): Defect B and regression tests.

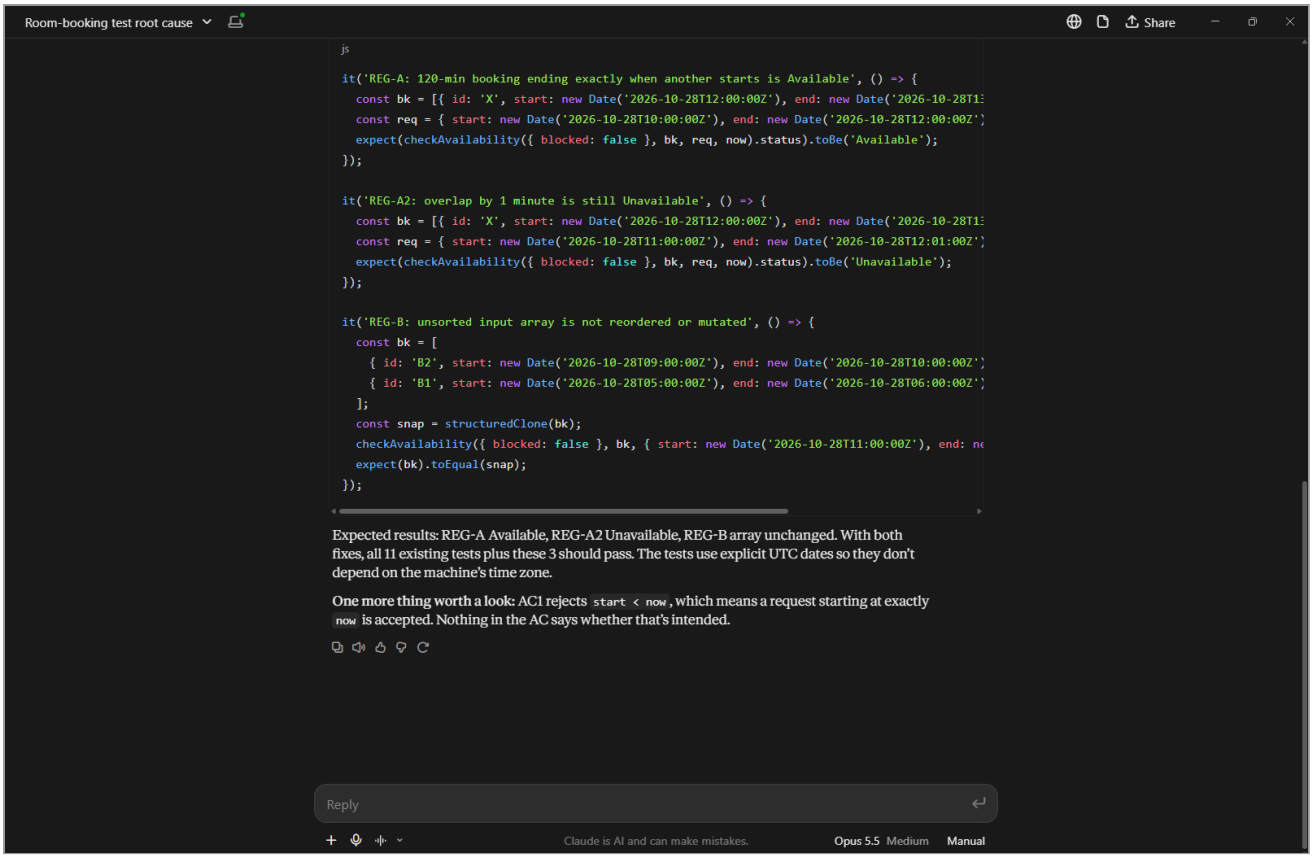


Figure 9. WITH skill (3/3): expected results and AC1 note.

4. Repair and regression (Task #06)

Commit	Change	Result
1d19bed	FIX-A: <= → < in `overlaps()`	1 failed / 10 passed
8af4066	FIX-B: sort a copy `[...bookings]`	**11 passed / 11**

Diff (evidence/code.diff)

```
diff --git a/src/availability.js b/src/availability.js
index dabfd11..c931f0f 100644
--- a/src/availability.js
+++ b/src/availability.js
@@ -27,8 +27,8 @@ export function checkAvailability(room, bookings, request, now) {
   if (room.blocked) return { status: 'Unavailable', reason: 'Room is blocked' };

   // AC4: overlaps
-  bookings.sort((a, b) => a.start - b.start); // BUG-B: sorts caller's array in place
-  for (const b of bookings) {
+  const sorted = [...bookings].sort((a, b) => a.start - b.start); // FIX-B: sort a copy, input unt
ouched
+  for (const b of sorted) {
     if (overlaps(b, request)) return { status: 'Unavailable', reason: 'Overlaps an existing bookin
g' };
   }

@@ -37,5 +37,5 @@ export function checkAvailability(room, bookings, request, now) {
 }

 export function overlaps(a, b) {
-  return a.start <= b.end && b.start <= a.end; // BUG-A: touching endpoints counted as conflict
+  return a.start < b.end && b.start < a.end; // FIX-A: strict comparison, touching endpoints allow
ed
 }
```

After FIX-A (evidence/02-after-fix-A.txt)

```
RUN  v2.1.9 /home/claude/smart-campus

 ❯ tests/availability.test.js (11 tests | 1 failed) 15ms
   × AC5 — no mutation of booking data > TC11 keeps bookings array order and content unchanged 10ms
     → expected [ { id: 'B1', …(2) }, …(1) ] to deeply equal [ { id: 'B2', …(2) }, …(1) ]

──────── Failed Tests 1 ────────

 FAIL  tests/availability.test.js > AC5 — no mutation of booking data > TC11 keeps bookings array o
rder and content unchanged
AssertionError: expected [ { id: 'B1', …(2) }, …(1) ] to deeply equal [ { id: 'B2', …(2) }, …(1) ]
… (full log in evidence/02-after-fix-A.txt)
```

Final run (evidence/03-final-test-output.txt)

```
RUN  v2.1.9 /home/claude/smart-campus

 ✓ tests/availability.test.js (11 tests) 7ms

 Test Files  1 passed (1)
      Tests  11 passed (11)
   Start at  18:26:28
   Duration  276ms (transform 43ms, setup 0ms, collect 40ms, tests 7ms, environment 0ms, prepare 54
ms)
```

5. Conclusion

Both answers found the same two root causes, so the skill did not make the AI smarter. What it changed was discipline: the answer followed the defect | evidence | root cause | fix | test structure, kept the patch to one line per bug, and marked the TC04 explanation as a hypothesis instead of stating it as fact. The tests prove that touching bookings (11:00 after a 10:00–11:00 booking) are now allowed while real overlaps are still rejected, and that checkAvailability no longer reorders the caller's bookings array. They do not prove correctness for multiple rooms, time zones other than the fixed one, or UI behaviour on mobile (AC6 was checked by hand only). One limitation of AI-assisted debugging: the AI only sees what I paste, so it can claim a fix works without running anything — only real terminal output counts as evidence.

Appendix A. Source code after fixes (src/availability.js)

```
// Smart Campus — room availability (Practice #06/#07/#08)
// User story: As a student, I want to check a room's availability
// so that I can choose a valid booking time.

export const MAX_DURATION_MIN = 120;

/**
 * Check whether a single room is available for a requested slot.
 * @param {{blocked:boolean}} room
 * @param {{start:Date,end:Date}[]} bookings  existing bookings (must NOT be modified)
 * @param {{start:Date,end:Date}} request
 * @param {Date} now  fixed clock (injected for deterministic tests)
 * @returns {{status:'Available'|'Unavailable', reason?:string}}
 */
export function checkAvailability(room, bookings, request, now) {
  const { start, end } = request;

  // AC1: start must be before end, and not in the past
  if (start >= end) return { status: 'Unavailable', reason: 'Start must be before end' };
  if (start < now) return { status: 'Unavailable', reason: 'Request is in the past' };

  // AC2: maximum duration
  const minutes = (end - start) / 60000;
  if (minutes > MAX_DURATION_MIN) return { status: 'Unavailable', reason: 'Longer than 120 minutes'
};

  // AC3: blocked room
  if (room.blocked) return { status: 'Unavailable', reason: 'Room is blocked' };

  // AC4: overlaps
  const sorted = [...bookings].sort((a, b) => a.start - b.start); // FIX-B: sort a copy, input unto
uched
  for (const b of sorted) {
    if (overlaps(b, request)) return { status: 'Unavailable', reason: 'Overlaps an existing bookin
g' };
  }

  // AC5
  return { status: 'Available' };
}

export function overlaps(a, b) {
  return a.start < b.end && b.start < a.end; // FIX-A: strict comparison, touching endpoints allowe
d
}
```

Appendix B. Tests (tests/availability.test.js)

```
import { describe, it, expect, beforeEach } from 'vitest';
import { checkAvailability } from '../src/availability.js';

// Fixed clock: 2026-10-27 09:00 (Asia/Almaty, UTC+5)
const NOW = new Date('2026-10-27T09:00:00+05:00');
const t = (hhmm) => new Date(`2026-10-27T${hhmm}:00+05:00`);

let room, bookings;
beforeEach(() => {
  // fresh fixtures for every test (fictional bookings)
  room = { id: 'A-301', blocked: false };
  bookings = [
    { id: 'B2', start: t('14:00'), end: t('15:00') },
    { id: 'B1', start: t('10:00'), end: t('11:00') },
  ];
});

describe('AC1 — valid time range', () => {
  it('TC01 rejects start == end', () => {
    expect(checkAvailability(room, bookings, { start: t('12:00'), end: t('12:00') }, NOW).status).t
oBe('Unavailable');
  });
  it('TC02 rejects start > end', () => {
    expect(checkAvailability(room, bookings, { start: t('13:00'), end: t('12:00') }, NOW).status).t
oBe('Unavailable');
  });
  it('TC03 rejects a request in the past', () => {
    expect(checkAvailability(room, bookings, { start: t('08:00'), end: t('08:30') }, NOW).status).t
oBe('Unavailable');
  });
});

describe('AC2 — max duration 120 min', () => {
  it('TC04 allows exactly 120 minutes', () => {
    expect(checkAvailability(room, bookings, { start: t('11:00'), end: t('13:00') }, NOW).status).t
oBe('Available');
  });
  it('TC05 rejects 121 minutes', () => {
    expect(checkAvailability(room, bookings, { start: t('11:30'), end: t('13:31') }, NOW).status).t
oBe('Unavailable');
  });
});

describe('AC3 — blocked room', () => {
  it('TC06 rejects a blocked room', () => {
    room.blocked = true;
    expect(checkAvailability(room, bookings, { start: t('12:00'), end: t('13:00') }, NOW).status).t
oBe('Unavailable');
  });
});

describe('AC4 — overlaps', () => {
  it('TC07 rejects a real overlap', () => {
    expect(checkAvailability(room, bookings, { start: t('10:30'), end: t('11:30') }, NOW).status).t
oBe('Unavailable');
  });
  it('TC08 allows a booking that starts exactly when another ends', () => {
    expect(checkAvailability(room, bookings, { start: t('11:00'), end: t('12:00') }, NOW).status).t
oBe('Available');
```

```
  });
  it('TC09 allows a booking that ends exactly when another starts', () => {
    expect(checkAvailability(room, bookings, { start: t('13:00'), end: t('14:00') }, NOW).status).t
oBe('Available');
  });
  it('TC10 allows a free slot', () => {
    expect(checkAvailability(room, bookings, { start: t('16:00'), end: t('17:00') }, NOW).status).t
oBe('Available');
  });
});

describe('AC5 — no mutation of booking data', () => {
  it('TC11 keeps bookings array order and content unchanged', () => {
    const snapshot = structuredClone(bookings);
    checkAvailability(room, bookings, { start: t('16:00'), end: t('17:00') }, NOW);
    expect(bookings).toEqual(snapshot);
  });
});
```

Appendix C. Optional second skill (regression-tester/SKILL.md)

```
---
name: regression-tester
description: Derive regression tests from acceptance criteria after a bug fix. Use when adding or r
eviewing Vitest tests for the Smart Campus room availability function.
---

# Regression Tester

## Procedure
1. List every acceptance criterion (AC1-AC6) and map each one to at least one test.
2. Use a fixed clock (inject `now`) - never call `new Date()` without arguments.
3. Create fresh fixtures in `beforeEach`; never share mutable data between tests.
4. Assert exact results (`'Available'` / `'Unavailable'`), not just truthiness.
5. Add boundary cases: equal start/end, exactly 120 min, 121 min, touching endpoints.
6. Assert non-mutation: deep-copy inputs before the call and compare after.
7. Run tests only if a terminal tool is available; otherwise say so.
8. Report unexecuted tests honestly - never claim "tests passed" without real output.

## Output
Table: AC | test ID | input | expected | executed? (yes/no)
```

Appendix D. package.json

```
{
  "name": "smart-campus",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "directories": {
    "test": "tests"
  },
  "scripts": {
    "test": "vitest run"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "devDependencies": {
    "vitest": "^2.1.9"
  },
  "type": "module"
}
```